HTML Structure and Semantics — From a Blank Page to a Web Page

Introduction to Web Technology — Week 2 (Module 2)

Index

- 1. From Blank Page to Web Page — the pep talk: what this module promises and why a white page is scarier than it looks.
- 2. HTML The Universal Mother Tongue — how one language got every browser on Earth to (mostly) agree.
- 3. A Guided Tour of a Real Page — peeking under the hood with DevTools, no mechanic's licence needed.
- 4. Anatomy of an Element — tags, attributes and content: the atoms of every page.
- 5. The Family Tree of a Document — parents, children, siblings; the drama of nested tags.
- 6. Semantic Tags Carry Meaning — why `<header>` beats a `<div>` wearing a name badge.
- 7. The Skeleton Every Page Shares — the boilerplate you will type a thousand times.
- 8. Headings and Paragraphs — giving your text an outline instead of a wall of words.
- 9. Lists — bullets, numbers, and the lesser-known definition list.
- 10. Links — the "hyper" in hypertext: teleporting across pages, sites and sections.
- 11. Cheat Sheet — one table to rule them all.
- 12. Practice Questions — five brain-benders to make sure it stuck.

---

1. From Blank Page to Web Page

The Story So Far

Imagine you open your favourite newspaper portal. The page you get is not one neat block of text. It has headlines, dozens of images, advertisements, tables, columns, and maybe a login box. On a social media site you can even *upload* data (a photo, a post) back to the server, and that too happens through a web page. Also remember from earlier that all of this content need not come from a **single server**; the browser assembles it from several places.

So the fundamental question of this module is:

> How do we turn a **blank white page** into a **structured, styled, responsive page** that any browser can render?

What This Module Covers

Topic	What you will learn
HTML structure and semantics	How a document is built, and why the tags you pick matter
CSS fundamentals	Syntax, selectors, and how conflicting rules actually get resolved
The box model	The four layers that determine an element's real size on the page
Layout and responsive design	Positioning basics, then Flexbox and Grid

A few words on the second and third rows, since they explain *why* this module is ordered the way it is:

- Some styles are **browser defaults**, while users and developers can define **their own** styles. When they conflict, a set of resolution rules (the *cascade*) decides who wins.
- Screens differ in size and resolution from device to device. The box model tells us the *real* size an element occupies, which affects how the user experiences the content.

Learning Objectives

Objective	Goal
HTML Structure	Structure a page correctly with semantic HTML5 elements
Tables, Forms, Media	Build tables and forms, and embed images, audio, and video
CSS and Box Model	Style with selectors, the cascade, and the box model
Flexbox and Grid	Lay out a responsive page with Flexbox and Grid

HTML has far more detail than one module can exhaustively cover. The aim here is to give you the fundamentals and plenty of pointers; real fluency in HTML and CSS comes from **hands-on practice**, which your lab experiments will provide.

> **Note for Review** Think of a web page as a house. **HTML** is the walls, rooms and doors (structure), **CSS** is the paint and furniture (style), and **JavaScript** is the electricity (behaviour). This module starts with the walls.

2. HTML The Universal Mother Tongue

Why We Need One Language

Delivering information globally requires a **universally understood language**. Here "language" does not mean English versus French or Spanish. It means: once the content is fixed (say, in English), how do we make it show up *the same way* on very different platforms?

- a small mobile phone, an 11 or 13 inch laptop, or a big desktop workstation,
- Linux, Windows or another operating system,
- any of the many web browsers.

Despite this heterogeneity, we want the **presentation** of the information to be broadly consistent for every client. HTML is that shared publishing language.

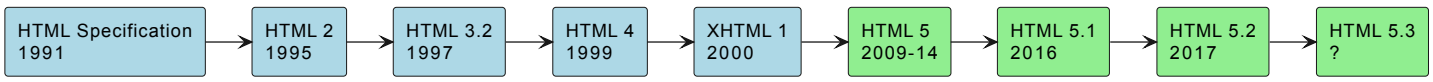
The slide puts it this way:

> *"To publish information for global distribution, one needs a universally understood language, a kind of publishing mother tongue that all computers may potentially understand. The publishing language used by the World Wide Web is HTML (from HyperText Markup Language)."* In 1998 the W3C froze the HTML specification at version 4.01.

Notice the word **Markup**. HTML works by *marking up* content with **tags** that tell the browser how a piece of content is to be treated: is this a heading, a paragraph, an image?

Timeline of HTML

Year	Milestone
1991	HTML Specification
1995	HTML 2
1997	HTML 3.2
1999	HTML 4 (W3C froze it at 4.01 in 1998, as quoted above)
2000	XHTML 1
2009 to 2014	HTML 5
2016	HTML 5.1
2017	HTML 5.2
?	HTML 5.3



Each version reflected growing interest, changing requirements and a growing set of applications. The W3C (World Wide Web Consortium) standardised HTML 4 around 1998, and from there the language kept evolving towards HTML5 and beyond, aiming at more capability and robustness.

What HTML5 Set Out To Do

1. **Specify unambiguously how browsers should deal with invalid markup.** Markup is simply the set of tags. If a page contains something the browser does not expect, the browser must not behave in a random, vendor-specific way; HTML5 defines how such errors are handled and reported so that all browsers react alike.
2. **Provide an open, non-proprietary programming framework** (via JavaScript) for creating rich web applications. This encourages **reuse of code**: you do not build an application from scratch, you can port and reuse existing components.
3. **Be backward compatible with the existing web.** Older pages and applications cannot simply be scrapped, so new features must coexist with old content.

The Golden Rule: HTML Describes Structure, CSS Describes Looks

> HTML only describes the **structural semantics** (the meaning) of the document. Information about how the content should *look* when displayed in the browser is best left to **CSS (Cascading Style Sheets)**.

Two jobs, two tools:

Job	Tool	Examples
Structure and meaning: where things go, what they are	**HTML**	this is a heading; this is the body; here comes an image, a table, an upload form
Presentation: how things look	**CSS**	bold or not, font size, colour, beautification

HTML lays out the *template* of the page: the heading portion, the body portion, where the text goes, where the image comes, and how a form should send a **file** versus **free text** back to the server. It handles all sorts of content: text, images, video, audio and tables.

> **Note for Review** Pro-Tip: This often appears on exams regarding *HTML vs CSS*. Remember that HTML answers "**what is this piece of content?**" while CSS answers "**how should it look?**". Even `<strong>` and `<em>` are about *meaning* (importance, emphasis); that they usually render bold and italic is only a default appearance.

---

3. A Guided Tour of a Real Page

Before studying the details, let us look at a typical page. The lecture uses a single self-contained demo file called *HTML Basics Demo* ("a one-page tour of the tags every web page starts with").

Opening It

- It is a plain `.html` file. Double-clicking it opens it in a browser (Edge, Chrome or any other). No server, no internet connection and no extra files are needed.
- The address bar shows the **path** to the file on the local disk (for example, on the C drive with several levels of directories), ending in the `.html` file. On real websites you are constantly opening `.html` resources in the same way.

Viewing the Code

Any browser lets you see the code behind the page:

Method	How
Developer Tools	Press `Ctrl + Shift + I`, or use the browser menu; select the HTML to see the whole code
View Page Source	Right click and choose *View page source*, or press `Ctrl + U` (`Cmd + U` on a Mac)

The DevTools panel shows an **Elements** tab with the document tree, and a **Styles** tab listing the CSS rules that apply to the selected element. In the Styles tab you will notice rules such as `box-sizing: border-box` and default rules labelled as coming from the **user agent stylesheet** (the browser's built-in defaults, for instance `html { display: block; }`). That is exactly the "browser defaults versus your own styles" idea from the introduction; we return to it in the CSS part of the module.

What the Demo Page Contains

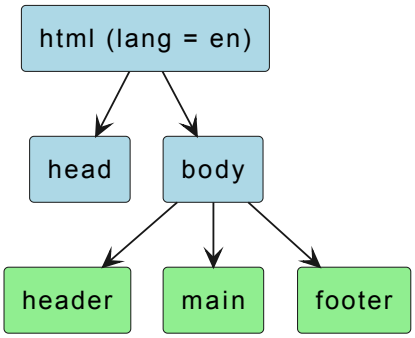
The page has a coloured header with a **navigation bar** (Headings, Lists, Table, Media, Form). These are **hyperlinks** that jump to the corresponding section of the *same page*, so you do not have to scroll. Below it:

Section	What it demonstrates
Headings and Paragraphs	`<h1>`, `<h2>`, `<h3>`, `<p>`; **bold and important** text with `<strong>`, *emphasised* text with `<em>`; a line break inside a paragraph with ` `; a horizontal rule with `<hr>` to separate ideas
Lists	Unordered list (`<ul>`) for items with no particular order; ordered list (`<ol>`) for sequential steps; description list (`<dl>`) for term and definition pairs
Table	`<table>` the whole table, `<tr>` a table row, `<th>` a header cell, `<td>` a regular data cell
Images	An image with a caption; in the demo the image is drawn from a tiny inline SVG so it works offline, but in a real project you write `<img src="photo.jpg" alt="...">` pointing at a real image file
Form	A simple form you fill in and submit, which uploads the data
Quick Reference sidebar	Written with the semantic `<aside>` tag: content related to, but separate from, the main flow of the page

The demo also shows the standard sequence for building a page:

1. Write the HTML (**structure**).
2. Style it with CSS (**style**).
3. Make it interactive with JavaScript (**behaviour**).

The DevTools view shows the overall shape of the demo file, which matches the skeleton we study in Section 7:



> The developer left a comment at the top of the file describing it as a single self-contained page for classroom demonstration. Comments like this are written between `<!--` and `-->` and are ignored by the browser.

The takeaway: HTML gives you **structural semantics**, meaning how a page is organised for different types of data (text, lists, tables, images, forms).

4. Anatomy of an Element

Tags, Braces and Scope

A **tag** is written inside angle brackets. In any programming language some keywords have a fixed, special meaning that the compiler or interpreter recognises; HTML tags play that role for the browser.

Every HTML file starts with this line, which declares it as an HTML5 document:

```
<!DOCTYPE html>
```

This is an exclamation mark, the word `DOCTYPE` in capitals, and `html` in lowercase, all inside angle brackets.

Think of C or C++ braces: an opening `{` and a closing `}` define the **scope** of what lies between them. HTML tags do the same job, in symmetric pairs:

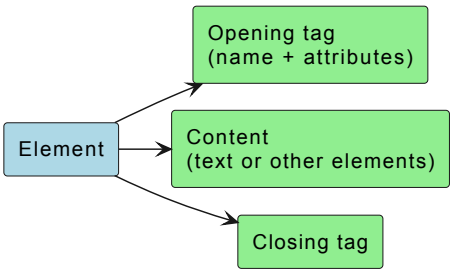
- `<html>` ... `</html>`
- `<head>` ... `</head>`
- `<body>` ... `</body>`

The **closing tag always has a forward slash** `/` after the opening angle bracket. Anything written between the pair belongs to that element.

The Parts of an Element

```
<a href="http://www.centralpark.com">Central Park</a>
```

Part	In the example above
Opening tag	`<a href="http://www.centralpark.com">`
Element name	`a`
Attribute	`href="http://www.centralpark.com"`
Content (may be text or other HTML elements)	`Central Park`
Closing tag	`</a>`



Empty Elements

Some elements have no content and therefore no closing tag. Example of an **empty element**:

```
<img src="file.gif" alt="something" />
```

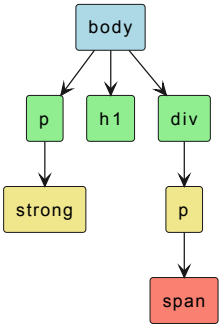
It has an element name (`img`), attributes (`src` , `alt`), and an **optional trailing slash** at the end.

> **Note for Review** Pro-Tip: A frequent exam point is the difference between a **tag** and an **element**. A *tag* is just the keyword in angle brackets. An *element* is the opening tag, its content and its closing tag **together**. (The demo page's description list says exactly this.)

5. The Family Tree of a Document

HTML is always structured **hierarchically**, so we borrow family vocabulary to describe how elements relate. Consider this fragment:

```
<body>
  <p>This is some <strong>text</strong></p>
  <h1>Title goes here</h1>
  <div>
    <p>This is <span>important</span></p>
  </div>
</body>
```

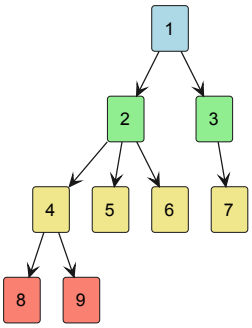


Term	Meaning	Example from the fragment
Parent	The element directly one level above	`body` is the parent of `p`, `h1` and `div`
Child	An element directly one level below	`p`, `h1` and `div` are children of `body`
Siblings	Elements sharing the same parent	`p`, `h1` and `div` are siblings
Descendants	Everything nested below, at any depth	`strong`, the inner `p` and `span` are descendants of `body`
Ancestors	Everything above, at any depth, up to the root	the `span`'s ancestors are the inner `p`, `div` and `body`

The **scope** of an element runs from its opening tag to its closing tag. Whatever sits between `<h1>` and `</h1>` (say, a `<span>`) is one level down, in the subtree of the `<h1>`.

Generalising: A Tree of Numbered Nodes

The lecture generalises the idea using a tree with numbered nodes. Let node 1 be the root. Its children are nodes 2 and 3. Node 2 has children 4, 5 and 6. Node 4 has children 8 and 9. Node 3 has child 7.



Writing $N(k)$ for "node k", the relations are:

$$N(2) = \text{Child}[N(1)] \qquad N(2) = \text{Parent}[N(4)]$$

The **descendants** of a node are everything in its subtree:

$$\text{Desc}(N(2)) = \{N(4), N(5), N(6), N(8), N(9)\} \qquad \text{Desc}(N(4)) = \{N(8), N(9)\}$$

and the **ancestors** of a node are everything on its path up to the root:

$$\text{Anc}(N(8)) = \{N(4), N(2), N(1)\}$$

Why does this matter? **Scoping is crucial for styling.** How the document is organised decides how we select elements later, and where we should limit the reach of a style rule.

> **Note for Review** Pro-Tip: This often appears on exams regarding *relationships in the DOM tree*. Remember that "child" means **one level** down but "descendant" means **any depth** down; every child is a descendant, but not every descendant is a child. The same goes for parent versus ancestor.

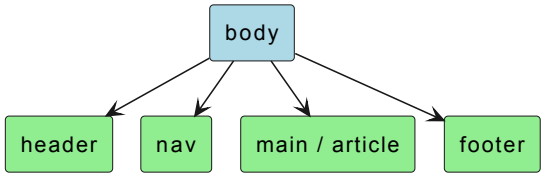
6. Semantic Tags Carry Meaning

It would be wrong to think of HTML tags as just a collection of boxes. Tags such as `<header>` and `<footer>` carry **specific meaning** about their role on the page.

The Same Page, Two Naming Schemes

How you might name it	What it actually means
`<div class="header">`	`<header>`
`<div class="nav">`	`<nav>`
`<div class="main">`	`<main>` or `<article>`
`<div class="footer">`	`<footer>`

The visual page is **identical** in both cases, but only the right-hand column is meaningful to **browsers, screen readers, and search engines**.



What Each Semantic Tag Says

- **`<header>`** is the heading portion of a page and has a very specific meaning; so does **`<footer>`**.
- **`<nav>`** is the navigation section: links to other pages, or links that jump to sections of the current page.
- **`<main>`** / **`<article>`** hold the primary content.
- **`<aside>`** holds content related to, but separate from, the main flow (like the Quick Reference sidebar in the demo).

Why Navigation Matters: Usability

How a page renders is directly tied to how the **user experiences** it. Two objective measures are the **number of clicks** and the **number of scrolls** needed to get something done. A very long page that forces endless scrolling is poor design. A navigation bar at the top that jumps straight to a section reduces the effort and the latency to complete a task, so the page feels richer and more interactive.

The `div` and the Class Handle

A `<div>` is a generic HTML element that **divides the page into segments** (scopes). By itself it carries no meaning. To make it useful we give it a `class` attribute:

```
<div class="header"> ... </div>
```

The class name is a **handle**. Later, a style sheet uses that handle to pick out this particular box and give it its own styling. If the header contains several elements, or the navigation bar has several parts, and each should look different, we need such handles to reach each one distinctly.

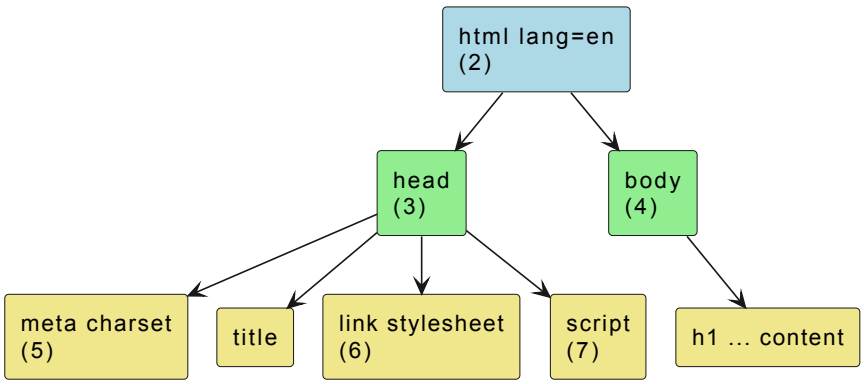
> **Note for Review** Pro-Tip: This often appears on exams regarding *class versus id*. A `class` name can be shared by many elements (that is the point: "style all of these alike"), whereas an `id` must be **unique** within a page. Use classes to group, ids to single out. And whenever a semantic tag exists (`header` , `nav` , `main` , `footer`), prefer it over a `div` with a class of the same name.

7. The Skeleton Every Page Shares

Every HTML document follows the same template:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Share Your Travels -- New York - Central Park</title>
    <link rel="stylesheet" href="css/main.css" />
    <script src="js/html5shiv.js"></script>
  </head>
  <body>
    <h1>Main heading goes here</h1>
    ...
  </body>
</html>
```

No.	Part	Role
1	`<!DOCTYPE html>`	Declares the document as HTML5; always the first line
2	`<html lang="en">`	The root element; the `lang` attribute states the language
3	`<head>`	Information *about* the page (not shown as page content)
4	`<body>`	The visible content of the page
5	`<meta charset="utf-8" />`	Standardises how text characters are represented
6	`<link rel="stylesheet" href="css/main.css" />`	Connects the page to its CSS file
7	`<script src="js/html5shiv.js"></script>`	Loads a JavaScript file when the page loads



The important structural point: `head` and `body` are **two distinct portions at the same level**. Their common parent is `html`, so they are **siblings**, both children of `html`. Inside `head` we can have several things: the character set, the title, links to other resources (such as the stylesheet), and scripts. Because we want the page to be **dynamic and interactive**, a script can be loaded here so it executes when the file loads; JavaScript code can also be embedded directly in the page, which we will see later.

> **Note for Review** Pro-Tip: This often appears on exams regarding *head vs body*. Remember that `<head>` holds **metadata and resources** (title, charset, stylesheet links, scripts) and `<body>` holds what the user **sees**. The page title appears in the browser tab, not on the page itself.

8. Headings and Paragraphs

Headings and paragraphs give text its **outline**.

- **Six heading levels** exist: `<h1>` through `<h6>`. Use them **in order**, and resist skipping from `<h2>` straight to `<h4>` just to get a smaller font. (Try `<h1>` versus `<h3>` in your hands-on session and see how each looks.)
- **`<p>`** marks a paragraph, used whenever you write free text. Browsers already add space above and below it automatically.
- **Extra spaces and line breaks** in your source collapse to a single space once the browser renders the page.
- **`<strong>` and `<em>`** signal real **importance** or **emphasis** to screen readers, not just bold or italic styling.

Why the last point matters: browsers can carry their own settings for styling content, which is one reason the experience differs slightly between browsers, even though our goal is a consistent experience everywhere. So `<strong>` and `<em>` are not merely "make it bold" or "make it italic"; they tell the renderer *how to treat* that part of the text.

Tag	Purpose	Note
`<h1>` to `<h6>`	Heading levels	Use in order; do not pick a level for its font size
`<p>`	Paragraph	Spacing above and below is automatic
`<strong>`	Strong importance	Usually displayed bold
`<em>`	Emphasis	Usually displayed italic
` `	Line break inside a paragraph	New line without a new paragraph
`<hr>`	Horizontal rule	A line separating ideas or sections

> **Note for Review** Pro-Tip: This often appears on exams regarding *whitespace in HTML*. Remember that pressing Enter five times in your source file does **not** create five blank lines. To force a new line use `<br>`, and to get real separation between blocks use separate `<p>` elements (and, later, CSS margins).

9. Lists

Lists organise related items. There are three kinds.

Kind	Tag	Marker	Use it for
Unordered list	`<ul>`	Bullets	Items with no particular order
Ordered list	`<ol>`	Numbers	Sequential steps
Description list	`<dl>`	Neither bullets nor numbers	Term and definition pairs

Every item in `<ul>` and `<ol>` is written with `<li>`.

Unordered List

```
<ul>
  <li><a href="index.html">Home</a></li>
  <li>About Us</li>
  <li>Products</li>
  <li>Contact Us</li>
</ul>
```

Browsers show this as a bulleted list, with *Home* as a clickable link. Notice that a **list item can contain other HTML elements** (here, a link).

Ordered List, with a Nested List

```
<ol>
  <li>Introduction</li>
  <li>Background</li>
  <li>My Solution
    <ol>
      <li>Methodology</li>
      <li>Results</li>
      <li>Discussion</li>
    </ol>
  </li>
  <li>Conclusion</li>
</ol>
```

Rendered result: 1 Introduction, 2 Background, 3 My Solution (with sub-items 1 Methodology, 2 Results, 3 Discussion), 4 Conclusion. The inner list is *inside* the third `<li>`, which is why it appears indented under "My Solution" and does not disturb the outer numbering.

Description List

A description list starts with `<dl>`. Each entry uses **`<dt>`** (description term, the first part) and **`<dd>`** (description details, the second part):

```
<dl>
  <dt>Tag</dt>
  <dd>A keyword in angle brackets, like p</dd>
  <dt>Element</dt>
  <dd>An opening tag, its content, and its closing tag together</dd>
</dl>
```

It is still a list, but neither of the unordered nor the ordered type.

> **Note for Review** Pro-Tip: This often appears on exams regarding *nested lists*. Remember that a nested `<ul>` or `<ol>` must be placed **inside an `<li>`** of the parent list, not between two `<li>` elements. Otherwise you get an unintended empty item and broken numbering.

---

10. Links

The **anchor element** `<a>` creates hyperlinks. It has a **destination** (the `href` attribute) and a **label** (its content), which can be text or an image.

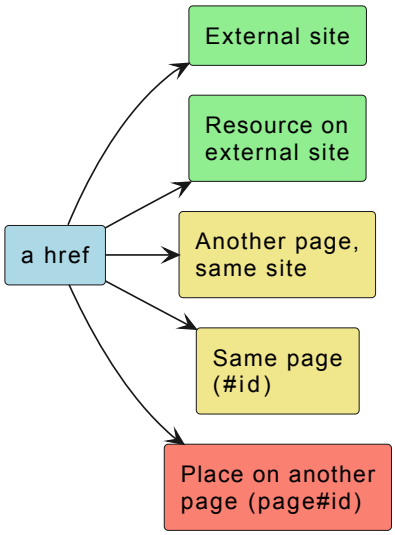
Label: Text or Image

```
<a href="http://www.centralpark.com">Central Park</a>
<a href="index.html"><img src="logo.gif" alt="logo" /></a>
```

- In the first, the destination is the URL and the label is text.
- In the second, the label is an **image**. Clicking it takes you to `index.html`. This is the familiar pattern of clicking the logo at the top-left corner of an institute or company site to return to its home page.

Where Can a Link Point?

Target	Example
Another site (external)	`<a href="http://www.centralpark.com">Central Park</a>`
A **resource** on an external site	`<a href="http://www.centralpark.com/logo.gif">Central Park</a>`
Another page on the **same site**	`<a href="index.html">Home</a>`
Another place on the **same page**	`<a href="#top">Go to Top of Document</a>`
A specific place on **another page**	`<a href="productX.html#reviews">Reviews for product X</a>`



Reading a URL Carefully

In `http://www.centralpark.com/logo.gif` the parts are:

- `www.centralpark.com` is the **server address** (resolved via DNS, from Module 1),
- `/logo.gif` is the **path** of the particular resource on that server.

So a link can point to the website itself, or to an *exact resource* on it, which the browser then fetches into your page.

Jumping Within a Page

To jump to a spot on the same page you need **two things**:

1. A **destination marker**: an element with an `id`, for example `<a id="top"></a>`, which defines the **anchor** for a link to another place on the same page.
2. A **link** to it: `href="#top"`, where the hash `#` is followed by that id.

Without a matching label (the `id`), the link has nowhere to go. To link to a section of *another* page, append the hash and id to the page name, as in `productX.html#reviews`. This is exactly how the navigation bar in the demo page jumps to sections. Get into the habit of reading the URLs you see while browsing: you will spot such `#` fragments and resource paths everywhere.

> **Note for Review** Pro-Tip: This often appears on exams regarding *relative versus absolute links*. Remember that `index.html` (no domain) is resolved **relative to the current page's location**, while `http://www.centralpark.com/...` is a full, absolute address. A `#` fragment never triggers a fresh page fetch when it points within the current page; it only scrolls to the target.

11. Cheat Sheet

Idea	Remember
HTML's job	Structure and meaning (**structural semantics**)
CSS's job	Presentation: colour, font, size, look and feel
Element	Opening tag + content + closing tag
Empty element	No content, no closing tag (for example `<img />`)
Scope	Everything between the opening and closing tag
Tree vocabulary	Parent, child, sibling, ancestor, descendant
First line	`<!DOCTYPE html>`
Two children of `html`	`head` (about the page) and `body` (the visible page)
Semantic tags	`header`, `nav`, `main`, `article`, `aside`, `footer`
Headings	`h1` to `h6`, in order, never for font size alone
Whitespace	Collapses to one space when rendered
Lists	`ul` (bullets), `ol` (numbers), `dl` with `dt` and `dd`
Links	`<a href="...">label</a>`; `#id` jumps to a place

12. Practice Questions

Try to answer these in your own words before checking your understanding against the notes.

1. **The invisible difference.** Two developers build visually identical pages. One uses `<div class="nav">`, `<div class="main">` and `<div class="footer">`; the other uses `<nav>`, `<main>` and `<footer>`. A visually impaired user browsing with a screen reader can jump straight to the main content on one page but not on the other. Which page is which, and what exactly is the reader using that the other page fails to provide? Would a search engine treat the two pages equally? Justify your answer using the idea of "meaning, not just boxes".

2. **Reading the tree.** Given the fragment below, list the parent, children, siblings, ancestors and descendants of the `<em>` element. Then suppose a style is meant to apply only to text inside the *second* `<section>`. Explain how the tree structure lets you limit the reach of that style, and what would go wrong if the second `<section>` were accidentally moved one level up, outside `<main>`.

```
<main>
  <section><p>One</p></section>
  <section><p>Two <strong>three <em>four</em></strong></p></section>
</main>
```

3. **The broken menu.** A student writes the following nested list and complains that the numbering shows 1, 2, 3, then an unexpected blank number, then the sub-items, then 5 for "Conclusion". Predict exactly what the browser displays, explain which rule of nesting was violated, and rewrite the code so that "Methodology" and "Results" appear indented beneath "My Solution".

```
<ol>
  <li>Introduction</li>
  <li>Background</li>
  <li>My Solution</li>
  <li><ol><li>Methodology</li><li>Results</li></ol></li>
  <li>Conclusion</li>
</ol>
```

4. **The 400-line page.** You are asked to build a single long tutorial page with 12 sections, plus a separate `glossary.html` page. Design the navigation so that (a) a reader can reach any section in one click from the top, (b) a reader on the glossary page can land directly on section 7 of the tutorial, and (c) every section has a "back to top" link. Write the `href` and `id` values you would use. Then explain what happens for the reader, and why, if one `id` is misspelled.

5. **Trading accessibility for looks.** A designer wants smaller text in the middle of an article and does this: an `<h1>` for the title, then an `<h4>` for each subsection, uses `<br><br>` between blocks of text in place of paragraphs, and writes important warnings in plain text made large. Identify at least three problems from the standpoint of structure and meaning, explain what each costs a screen reader user and a search engine, and propose the refactored structure. Which part of the fix belongs to HTML and which part must be delegated to CSS, and why?